RUNI – Algorithms – Homework 3

Due date: Sunday 30/11/2025

NOTE: the general homework guidelines published in homework 1 are valid for all homework assignments in this course.

Problem 1 (25 pts.):

For each of the following statements, decide whether it is true or false, and prove or refute accordingly.

1. Let $G = (V, E)$ be an **undirected**, and possibly unconnected, graph, and let $a, b \in V$. The number of trees in the forests returned by $DFS(a)$ and $DFS(b)$ is the same.

True. In an undirected graph there is a separate tree for every connected component.

2. Let $G = (V, E)$ be a **directed** graph and let $a, b \in V$. The numbers of trees in the forests returned by a $DFS(a)$ and $DFS(b)$ is the same.

False. Consider the graph $v_1 \rightarrow v_2$. Running $DFS(v_1)$ will yield one tree, while $DFS(v_2)$ will result in a forest with two trees.

3. Let G be an **undirected** graph. Assume you run $BFS(s)$. During the run, the vertex v marks the vertex u. In the run of $DFS(s)$ on G, that uses the same adjacency lists as the run of BFS, v will be marked before u.

False. For example, consider a cycle on the vertex set $V = \{s, a, b, u, v\}$ and $E = \{(s, a), (a, b), (b, u), (u, v), (v, s)\}$ where a appears before v in the list of s's neighbors.

4. Let G be an **undirected** graph that has a Hamiltonian cycle. For every vertex $v \in V$, there exists a run of $DFS(v)$ in which the resulting **directed** spanning tree has a single vertex with an out-degree of 0.

True. Assume the Hamiltonian cycle is $v_1, \ldots, v_n, v_1$. For every vertex v_i, assume the first vertex in the adjacency list is v_{i+1} , and for v_n it is v_1. From any vertex v_i, in the matching run of $DFS(v_i)$, the resulting spanning tree is a path containing all vertices, meaning only v_{i-1} has an our-degree of 0.

5. Let $G = (V, E)$ be an **undirected** graph with $|V| = 4$. Some run of DFS produces a spanning tree in which the root has degree 2.
Claim: $|E| \leq 4$.

True. Let V={a,b,c,d}. Assume that a run of DFS(a) produces a tree in which a has degree 2.

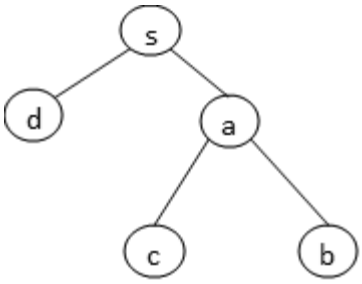
Assume by contradiction that $|E| > 4$. This implies that every node in the graph has degree at least 2. Consider a run of DFS(a). Assume w.l.o.g., that (ab) is the first edge to be traced in DFS(a). Since degree(b)≥2, it has at least one neighbor among c and d. If both c and d are neighbors of b, then both will be marked before the run backtracks to s. If only one of c and d is a neighbor of b, w.l.o.g., c, then, since degree(d)≥2, node d must be connected to a and d, therefore, the path b-c-d exists. We conclude that both c and d are connected to b by a path that does not include a, therefore, both will be marked before the run backtracks to a.

Problem 2 (15 pts):

1) Let $G_{2,4}$ be the De-Bruijn graph for $n = 4, \Sigma = \{0,1\}$. We perform $BFS(100)$ (namely, starting from the vertex 100). How many vertices will have a label of 2 at the end of the run? Justify your answer.

The two edges leaving the vertex 100 are (100,000) and (100,001). Therefore, the vertices 000 and 001 are labeled 1. The two edges leaving the vertex 000 are (000,000) and (000,001). Since both 000 and 001 are labeled 1 already, none of them is labeled 2. The two edges leaving the vertex 001 are (001,010) and (001,011). Their endpoints 010 and 011 are labeled 2.

2) Let G be an undirected graph, where $V = \{a, b, c, d, s\}$. $DFS(s)$ produces the following spanning tree.



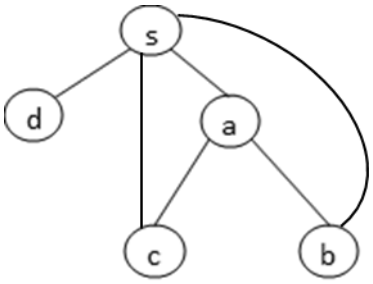
Draw a graph G that fulfills the above description and has a maximal number of edges. For every $(v, u) \notin E$, justify why this edge cannot be in E.

There are 6 edges to be considered, from them 4 can't be in the graph:

- (a, d) – in $DFS(s)$ either a would mark d or the other way around.
- (b, c) – in $DFS(s)$ we would have gotten one of the following sub-paths: $a -> b -> c$ or $a -> c -> b$
- (c, d) – in $DFS(s)$ either c would mark d or the other way around.
- (b, d) – in $DFS(s)$ either b would mark d or the other way around.

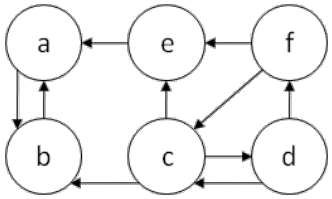
All of those scenarios are a contradiction to the given $DFS(s)$.

The edges (b, s), (c, s) can be added and form this graph:



Problem 3 (20 pts.):

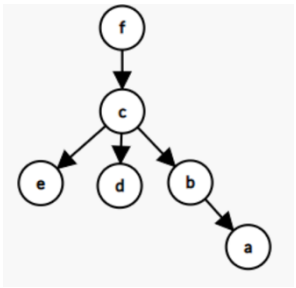
Consider the following directed graph G and let G' denote the underlying graph of G.



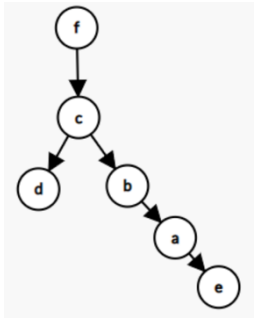
Assume that vertices in the adjacency list appear in alphabetic order. For example, c's list is $b \rightarrow d \rightarrow e$.

1) Run $DFS(f)$ on G and draw the resulting spanning tree of G.
2) Run $DFS(f)$ on G' and draw the resulting spanning tree of G'.
3) Run $BFS(d)$ on G and draw the resulting spanning tree of G.
4) Run $BFS(d)$ on G' and draw the resulting spanning tree of G'.

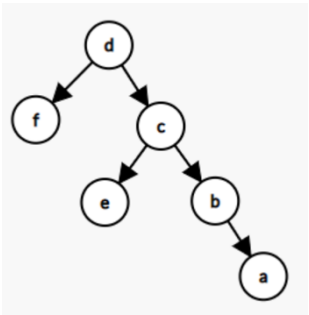
1)



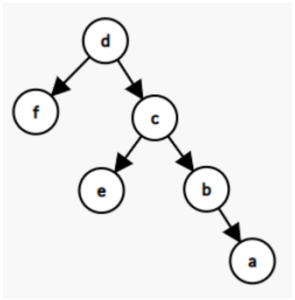
2)



3)



4)



Problem 4 (20 pts.):

A root in a directed graph $G = (V, E)$ is a vertex $r \in V$ such that there is a directed path from r to any other vertex $v \in V$.
Let $G = (V, E)$ be a directed graph. Describe an $O(|V| + |E|)$ time algorithm that finds all the roots in G or states that "a root does not exist". Analyze the running time of your algorithm and explain why it is correct.

Directions: As a first step, use the algorithm for finding a single root in a directed graph (this algorithm was taught in the practical session).

Algorithm:

- Run the algorithm for finding a root (if exists) in G (see recitation 4). Let r be a root in G.
- Build a new graph $H = (V, E')$ such that $E' = \{(u, v) | (v, u) \in E\}$ (in the graph H all edge directions are reversed). Clearly if a path from u to v exists in G, a path from v to u exists in H.
- Run $BFS(r)$
- r's descendants in the *BFS* tree are the addional roots of G.

Complexity:
$O(|V| + |E|)$ Since the algorithm performs at most 3 runs of DFS or BFS on a graph with $|V|$ vertices and $|E|$ edges.

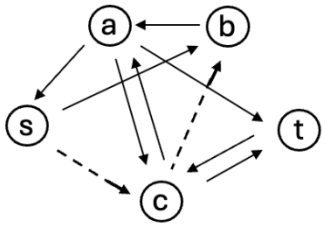
Correctness:
The idea is to find one root r and then (using BFS on the reversed graph) all the vertices $u \in V$ from which there is a path to r.
Claim: if $v \in V$ is a root, then $u \in V$ is also a root if and only if there is a directed path from u to v.
Proof:
→ Assume there is a directed path from u to v in G. Since v is a root in G, there is a path from v to every $w \in V$, and therefore from u to every $w \in V$ (through v).
← If there isn't a directed path from u to v, then clearly u isn't a root.

Problem 5 (20 pts.):

Let $G = (V, E)$ be an unweighted, simple directed graph where $s \in V$ is a source vertex and $t \in V$ is a target vertex ($s \neq t$). The graph presents a network of roads. As a result of floods, some of the roads, given by the edge set A, were closed. Suggest an algorithm that decides which road (only one) $e \in A$, should be reopened, so that the shortest path from s to t is as short as possible.

Describe the algorithm formally, explain in several lines its correctness (state the main claim in the proof, but no need to prove it), state and justify the time complexity.

Example: In the given graph, for $A = \{(s, c), (c, b)\}$ if (s, c) is selected to reopen, then the shortest path would be 2. If (c, b) is selected, the shortest path would be 3.



The important observation is that we are looking for a shortest path from s to t that includes at most one edge in A.

Algorithm:

1. Build a graph $G' = (V', E')$. Such that:
   $V' = V_1 \cup V_2$ where $V_i = \{v_i : v \in V\}$.
   $E' = \{(v_1, u_1) : (v, u) \in E \backslash A\} \cup \{(v_2, u_2) : (v, u) \in E \backslash A\} \cup \{(v_1, u_2) : (v, u) \in A\}$
2. Run $BFS(s_1)$ on G'. Keep for every vertex in the returned spanning tree a pointer to its parent vertex.
3. If t_1 was labeled before t_2, then no road is needed to be opened. If t_2 was labeled before t_1, then return the (single) edge from A in the path. This is the edge connecting V_1 and V_2, it can be detected by tracing the path backward

The algorithm builds a graph consisting of two copies of the vertices. Each of the copies includes all the edges that are not in A, and the two copies are connected by the edges in A. By the construction, every path from s_1 to v_1 includes no edges from A, and every path from s_1 to v_2 includes exactly one edge from A.

Claim: There is a path of length k from s to t that includes one edge from A, in G, if and only if there is a path of length k from s_1 to t_2 in G'. In particular, the shortest path from s_1 to t_2 in G' corresponds to the shortest path from s to t in G that includes one edge from A.

Time complexity:

In step 1, the algorithm builds the graph G'. The size of G' is linear in the size of G. Specifically, $|V'| = 2|V|, |E'| \leq 2|E|$. Therefore, building G' takes $O(|E| + |V|)$.

In step 2, the algorithm runs BFS on the graph G'. It takes $O(|V'| + |E'|) = O(|V| + |E|)$.

Step 3 takes $O(|V'| + |E'|) = O(|V| + |E|)$ (check which on between t_1 and t_2 was labeled first and in the case of it being t_2, backtrack to retrieve the edge from A.

All together, the time complexity is $O(|V| + |E|)$.